

AEGIS Project

Stage-N: Hybrid Quantum-Classical Token Execution Layer

Formal Specification for RiftLang Protocol Stack

Version 1.0 - Technical Specification Document

OBINexus Computing Division

Toolchain: `riftlang.exe` → `.so.a` → `rift.exe` → `gosilang`

July 12, 2026

Contents

1	Executive Summary	4
2	Stage Evolution Framework	4
3	Core Architecture and Purpose	4
3.1	Fundamental Design Principles	4
3.2	Integration Points	5
4	Quantum Token Specification	5
4.1	Core Quantum Token Definition	5
4.2	Extended Quantum Token Attributes	5
4.3	Quantum Token Memory Alignment	6
5	Classical Resolution Operator	6
5.1	Collapse Operator Definition	6
5.2	Piecewise Collapse Logic	6
5.3	State Transition Matrix	7
6	Memory-Governed Quantum Parser	8
6.1	Hybrid Token Grammar	8
6.2	Temporal Memory State Management	8
6.3	Parser State Machine	9
7	Governance Constraint Declarations	9
7.1	Core Governance Rules	9
7.2	Resource Management Constraints	10
7.3	Security and Validation Rules	10
8	Integration with AEGIS Phase Architecture	11
8.1	Phase I - Matrix Parity Integration	11
8.2	Phase II - Token Stream Management	12
8.3	Phase III - Planck Verification	12
9	Runtime Execution Model	13
9.1	Dual-Mode Execution Pipeline	13
9.2	Context Switching Protocol	13
10	Example Usage Patterns	14
10.1	Basic Quantum Token Operations	14
10.2	Hybrid Computation Pattern	14
11	Formal Verification Properties	15
11.1	Safety Properties	15
11.2	Liveness Properties	16
12	Performance Specifications	16

13 Firmware Integration Guidelines	16
13.1 Git-RAF Integration Points	16
14 Conclusion and Future Extensions	17
A Glossary of Terms	17
B References	18

1 Executive Summary

This document formalizes Stage-N of the RiftLang Protocol Stack, establishing the critical interface between quantum probabilistic computation and classical deterministic execution. Stage-N enables seamless transitions between quantum superposition states and classical computational models while maintaining strict AEGIS governance compliance throughout the quantum-classical boundary.

The specification defines standardized patterns for stages 0 through N+1 (currently implemented through Stage-7), providing a unified framework for quantum token management, collapse operations, and memory-governed parsing within the RIFT domain-specific language ecosystem.

2 Stage Evolution Framework

Stage Progression Model

- **Stage-0:** Token initialization and classical baseline
- **Stage-1:** Quantum extension introduction
- **Stage-2:** Entanglement protocol establishment
- **Stage-3:** Collapse operator implementation
- **Stage-4:** Memory governance integration
- **Stage-5:** Parser unification
- **Stage-6:** AEGIS phase alignment
- **Stage-7:** Full quantum-classical bridge deployment
- **Stage-N:** Dynamic stage instantiation
- **Stage-N+1:** Future extensibility framework

3 Core Architecture and Purpose

3.1 Fundamental Design Principles

Stage-N serves as the quantum-classical computation bridge within the RIFT DSL execution pipeline. The architecture enables:

1. **Quantum state preservation** during computation
2. **Deterministic resolution** when measurement occurs
3. **Governance enforcement** at state transitions
4. **Memory-bounded execution** with Planck-scale constraints

3.2 Integration Points

```

1  @stage_interface[N] {
2      input_stages: [N-1, quantum_init]
3      output_stages: [N+1, classical_exec]
4      governance_hooks: AEGIS_Phase_I_III
5      memory_model: quantum_foam_temporal
6      compliance_level: STRICT
7  }

```

Listing 1: Stage-N Integration Architecture

4 Quantum Token Specification

4.1 Core Quantum Token Definition

The fundamental quantum token represents a qubit in superposition state with complex amplitude coefficients.

```

1  @token[quantum] qbit superposition( $\alpha$ ,  $\beta$ ) → QINT
2  Where:
3       $\alpha$  in C : Complex amplitude for  $|0\rangle$  state
4       $\beta$  in C : Complex amplitude for  $|1\rangle$  state
5      Constraint:  $|\alpha|^2 + |\beta|^2 = 1$  (normalization)

```

Listing 2: Quantum Token Base Definition

4.2 Extended Quantum Token Attributes

```

1  @token_extension[quantum] {
2      # Dirac notation representation
3      bra_ket_notation:  $|\psi\rangle = \alpha |0\rangle + \beta |1\rangle$ 
4
5      # Normalization enforcement with tolerance
6      amplitude_norm: enforce( $|\alpha|^2 + |\beta|^2 = 1.0 \pm \epsilon$ )
7      Where:  $\epsilon = 10^{-15}$  (machine  $\epsilon$ )
8
9      # Decoherence threshold at Planck scale
10     decohere_threshold:  $\tau_{\text{planck}} = 5.39 \times 10^{-44}$  seconds
11
12     # Entanglement tracking
13     entanglement_flag: bool
14     entanglement_partners: QINT[] (max_size = 6)
15
16     # Phase coherence bounds
17     phase_coherence:  $\phi$  in  $[0, 2\pi]$ 
18     phase_drift_rate:  $d\phi/dt \leq \pi/\tau_{\text{coherence}}$ 
19 }

```

Listing 3: Quantum Token Extensions

4.3 Quantum Token Memory Alignment

```

1  @memory_align[quantum] {
2      alignment: 8-qubit boundary
3      span_type: distributed_quantum_foam
4      coherence_window: planck_time
5      isolation_level: phase_locked
6
7      layout: {
8          |q0> |q1> |q2> |q3> |q4> |q5> |q6> |q7>
9          [amplitude_real][amplitude_imag]
10         [phase][entangle_mask][coherence]
11     }
12 }
```

Listing 4: Memory Alignment Specification

5 Classical Resolution Operator

5.1 Collapse Operator Definition

The collapse operator manages the quantum-to-classical transition under governance constraints.

```

1  @operator collapse {
2      input: QINT                                # Quantum integer in
3          superposition
4      condition: coherence ≥ PLANCK_THRESHOLD
5      output: INT                                # Classical deterministic
6          integer
7      audit: quantum_event_log                    # Governance audit trail
8
9      properties: {
10         irreversible: true
11         measurement_basis: computational
12         entropy_increase: Δ_S > 0
13     }
14 }
```

Listing 5: Collapse Operator Specification

5.2 Piecewise Collapse Logic

```

1  PROCEDURE quantum_classical_collapse(q: QINT, E: energy, m:
2      mass):
3      LET c2 = speed_of_light2
4      LET E_planck = ħ * ω
5
6      IF E2 ≤ m * c2:
7          # Low energy - classical behavior dominates
```

```

7      state := CLASSICAL_DETERMINISTIC
8      RETURN cast_to_int(q, method="measurement")
9
10     ELIF E > E_critical AND q.coherence ≥  $\tau_{\text{planck}}$ :
11         # High energy with coherence - forced collapse
12         TRIGGER collapse_event {
13             log: quantum_event_log
14             timestamp: current_planck_time
15             method: "forced_decoherence"
16         }
17         state := CLASSICAL_COLLAPSED
18         RETURN probabilistic_cast(q)
19
20     ELSE:
21         # Maintain quantum superposition
22         state := QUANTUM_SUPERPOSITION
23         EVOLVE q WITH hamiltonian(H)
24         RETURN q # Preserve quantum state
25
26     END IF
27 END PROCEDURE

```

Listing 6: Collapse Decision Tree

5.3 State Transition Matrix

```

1  @state_transition_matrix {
2      QUANTUM → CLASSICAL: {
3          trigger: measurement OR decoherence
4          probability:  $|\langle \psi | \phi \rangle|^2$ 
5          governance: collapse_contract
6          audit_level: MANDATORY
7      }
8
9      CLASSICAL → QUANTUM: {
10         trigger: superposition_gate
11         condition: coherence_budget > threshold
12         governance: quantum_init_contract
13         audit_level: STRICT
14     }
15
16     QUANTUM → QUANTUM: {
17         trigger: unitary_evolution
18         operator:  $U = \exp(-iHt/\hbar)$ 
19         governance: evolution_contract
20         audit_level: PERIODIC
21     }
22 }

```

Listing 7: Quantum-Classical State Transitions

6 Memory-Governed Quantum Parser

6.1 Hybrid Token Grammar

The parser must handle both quantum and classical tokens with appropriate type safety.

```

1  @parser[hybrid_quantum_classical] {
2      token_types: {
3          QINT      : quantum_integer[superposition]
4          INT       : classical_integer[deterministic]
5          FLOAT     : classical_float[ieee754]
6          BRA       :  $\langle\psi|$  quantum_state
7          KET       :  $|\psi\rangle$  quantum_state
8          QFLOAT    : quantum_float[superposition]
9      }
10
11     # Regex pattern with quantum extensions
12     parse_rules: R"/([QC])(INT|FLOAT|STATE)/gmi[tb]"
13     Where:
14         g: global matching
15         m: multiline quantum states
16         i: case-insensitive operators
17         t: top-down classical resolution
18         b: bottom-up quantum composition
19 }

```

Listing 8: Token Type Definitions

6.2 Temporal Memory State Management

```

1  @memory_state::quantum_foam {
2      lifetime: planck_time = 5.39 x 10-44 seconds
3      scope: local_superposition
4
5      allocation: {
6          classical_mode: align(4096_bits)
7          quantum_mode: align(8_qubits)
8          hybrid_mode: interleaved_coherent
9      }
10
11     persistence: {
12         coherent_duration:  $\tau_{\text{coherence}}$ 
13         decoherence_rate:  $\Gamma = 1/T_2$ 
14         error_threshold: 10-9
15     }
16
17     governance: {
18         max_entanglement_depth: 6
19         bell_state_limit: 4_pairs
20         gc_policy: phase_aware_collection
21     }

```



```
22 }
```

Listing 9: Quantum Memory Management

6.3 Parser State Machine

```

1  AUTOMATON quantum_parser {
2      states: {S_INIT, S_QUANTUM, S_CLASSICAL, S_COLLAPSE,
3              S_MEASURE}
4
5      transitions: {
6          S_INIT → S_QUANTUM:
7              condition: detect_superposition_token()
8              action: init_quantum_context()
9
10         S_QUANTUM → S_COLLAPSE:
11             condition: coherence < PLANCK_THRESHOLD
12             action: prepare_collapse()
13
14         S_COLLAPSE → S_CLASSICAL:
15             condition: collapse_complete()
16             action: emit_classical_token()
17
18         S_QUANTUM → S_MEASURE:
19             condition: measurement_operator()
20             action: von_neumann_projection()
21
22         S_MEASURE → S_CLASSICAL:
23             condition: measurement_complete()
24             action: emit_measured_value()
25     }
26
27     error_states: {
28         E_COHERENCE_LOST: recovery = forced_collapse
29         E_ENTANGLE_VIOLATION: recovery = isolate_subsystem
30         E_MEMORY_OVERFLOW: recovery = quantum_gc
31     }
32 }
```

Listing 10: Quantum Parser Automaton

7 Governance Constraint Declarations

7.1 Core Governance Rules

```

1  @gov_rule::collapse_contract {
2      requires: {
3          coherence ≥ planck_threshold
4          entanglement_depth ≤ max_allowed
5      }
6  }
```

```

5         quantum_budget > operation_cost
6         audit_trail.enabled = true
7     }
8
9     prohibits: {
10         superposition_state > max_density_matrix_size
11         concurrent_measurements > 1
12         phase_drift >  $\pi/4$ 
13         untracked_entanglement = true
14     }
15
16     audit: {
17         log_destination: quantum_event_log
18         retention_period: 7_stages
19         cryptographic_seal: SHA3-256
20         immutability: blockchain_anchored
21     }
22 }

```

Listing 11: Collapse Contract Governance

7.2 Resource Management Constraints

```

1 @gov_rule::quantum_resource_management {
2     allocation_policy: {
3         max_qubits_per_token: 16
4         max_entangled_pairs: 8
5         decoherence_budget: 1000_planck_times
6         memory_quota: 1MB_quantum_foam
7     }
8
9     cleanup_policy: {
10         auto_collapse_timeout: 100_planck_times
11         garbage_collection: phase_aware
12         memory_reclaim: immediate
13         entanglement_pruning: depth_first
14     }
15
16     cost_model: {
17         superposition_cost: 0.1_per_qubit_per_cycle
18         entanglement_cost: 0.3_per_pair
19         measurement_cost: 0.2_per_operation
20         coherence_maintenance: 0.05_per_planck_time
21     }
22 }

```

Listing 12: Quantum Resource Governance

7.3 Security and Validation Rules

```

1  @gov_rule::quantum_security {
2      validation: {
3          state_vector_normalization: continuous
4          no_cloning_enforcement: strict
5          basis_state_verification: periodic(10_cycles)
6          bell_inequality_check: on_entanglement
7      }
8
9      access_control: {
10         quantum_state_read: privileged_only
11         collapse_trigger: authorized_operators
12         entanglement_create: rate_limited(10/sec)
13         phase_manipulation: governance_approved
14     }
15
16     integrity: {
17         checksum_algorithm: quantum_hash_SHA3Q
18         tamper_detection: bell_inequality_test
19         audit_trail: immutable_quantum_ledger
20         replay_protection: nonce_per_operation
21     }
22 }

```

Listing 13: Quantum Security Governance

8 Integration with AEGIS Phase Architecture

8.1 Phase I - Matrix Parity Integration

```

1  INTEGRATION matrix_parity_bridge {
2      quantum_to_fft: {
3          INPUT: QINT[superposition]
4          PROCESS:
5              1. Extract amplitude vectors ( $\alpha$ ,  $\beta$ )
6              2. Map to FFT basis:  $F(|\psi\rangle) = \text{Sum}(\alpha_k * e^{(2*\pi*ijk/N)})$ 
7              3. Apply parity constraints from Phase I
8              4. Verify matrix eigenvalue stability
9          OUTPUT: FFT_MATRIX[classical]
10     }
11
12     governance: {
13         parity_check: R"/[01]{8}/g"
14         matrix_alignment: 8x8_quantum_block
15         eigenvalue_threshold:  $|\lambda| < 1.0$ 
16     }
17 }

```

Listing 14: Matrix Parity Bridge

8.2 Phase II - Token Stream Management

```

1  INTEGRATION token_stream_quantum {
2      stream_mode: {
3          classical: sequential_ordered
4          quantum: parallel_superposed
5          hybrid: context_switched
6      }
7
8      synchronization: {
9          barrier: quantum_measurement_point
10         ordering: causal_cone_preservation
11         latency: ≤ coherence_window
12     }
13
14     buffering: {
15         quantum_buffer: circular_phase_locked
16         classical_buffer: FIFO_deterministic
17         transition_buffer: copy_on_collapse
18     }
19 }

```

Listing 15: Token Stream Integration

8.3 Phase III - Planck Verification

```

1  INTEGRATION planck_verification {
2      collapse_window: {
3          detection: coherence < PLANCK_THRESHOLD
4          action: enforce(collapse_contract)
5          verification: cryptographic_proof
6          timing: exact_planck_time
7      }
8
9      quantum_classical_boundary: {
10         transition_log: {
11             timestamp: planck_time_resolution
12             state_before:  $|\psi\rangle$ 
13             state_after: classical_value
14             entropy_change:  $\Delta_S$ 
15             information_preserved:  $I = -\text{Sum}(p \log p)$ 
16         }
17     }
18
19     entanglement_boundary: {
20         max_distance: 6_hops
21         isolation_enforcement: bell_state_collapse
22         audit_requirement: full_trace
23         correlation_preservation: EPR_compliant
24     }
25 }

```

Listing 16: Planck Scale Verification

9 Runtime Execution Model

9.1 Dual-Mode Execution Pipeline

```

1 PIPELINE rift_stage_n_execution {
2     MODE classical {
3         stages: tokenize → parse → validate → execute
4         memory: sequential_4096_aligned
5         concurrency: mutex_protected
6         error_handling: exception_based
7     }
8
9     MODE quantum {
10        stages: superpose → entangle → evolve → measure
11        memory: distributed_8qubit_foam
12        concurrency: phase_locked_parallel
13        error_handling: decoherence_recovery
14    }
15
16    MODE hybrid {
17        stages: detect_context → switch_mode → process →
18            reconcile
19        memory: adaptive_alignment
20        concurrency: quantum_classical_barrier
21        error_handling: graceful_degradation
22    }
23
24    performance: {
25        classical_throughput: 106 ops/sec
26        quantum_coherence: 1000 x planck_time
27        transition_overhead: < 1us
28    }
29 }
```

Listing 17: Stage-N Execution Pipeline

9.2 Context Switching Protocol

```

1 PROTOCOL context_switch {
2     classical_to_quantum: {
3         save_classical_state()
4         init_quantum_registers()
5         prepare_superposition()
6         verify_coherence()
7         enable_quantum_operations()
8     }
9 }
```

```

8     }
9
10    quantum_to_classical: {
11        prepare_measurement()
12        collapse_superposition()
13        extract_classical_value()
14        cleanup_quantum_resources()
15        restore_classical_context()
16    }
17
18    switch_overhead: {
19        time_cost: 0(n_qubits)
20        memory_cost: 0(2^n_qubits)
21        governance_cost: 0(audit_depth)
22    }
23 }

```

Listing 18: Quantum-Classical Context Switch

10 Example Usage Patterns

10.1 Basic Quantum Token Operations

```

1  # Initialize quantum token in superposition
2  @quantum
3  let q_value: QINT = superposition(0.707, 0.707) # |+> state
4
5  # Entangle two quantum tokens
6  @quantum
7  let q_pair: (QINT, QINT) = entangle(q_value, q_other)
8
9  # Conditional collapse based on coherence
10 @hybrid
11 if coherence(q_value) < PLANCK_THRESHOLD {
12     let classical_result: INT = collapse(q_value)
13     process_classical(classical_result)
14 } else {
15     evolve_quantum(q_value, hamiltonian)
16 }
17
18 # Measurement with basis selection
19 @quantum
20 let measured: INT = measure(q_value, basis="X")

```

Listing 19: Quantum Token Usage Examples

10.2 Hybrid Computation Pattern

```

1  @hybrid_algorithm grover_search {

```

```

2      # Classical preprocessing
3      let dataset: INT[] = load_classical_data()
4      let target: INT = define_search_target()
5
6      # Quantum acceleration phase
7      @quantum {
8          let q_register: QINT[] = init_superposition(size=log2(
9              dataset.length))
10
11          repeat sqrt(dataset.length) times {
12              apply_oracle(q_register, target)
13              apply_diffusion(q_register)
14
15              if coherence_degraded(q_register) {
16                  refresh_quantum_state(q_register)
17              }
18
19          let result_index: INT = measure_all(q_register)
20      }
21
22      # Classical verification
23      @classical {
24          verify_result(dataset[result_index], target)
25          return dataset[result_index]
26      }
27 }

```

Listing 20: Hybrid Quantum-Classical Algorithm

11 Formal Verification Properties

11.1 Safety Properties

```

1  PROPERTY quantum_safety {
2      # G1: Normalization is always maintained
3      [] (forall q: QINT . |amplitude(q)|2 = 1.0 +/- ε)
4
5      # G2: No-cloning theorem is preserved
6      [] (not exists operation . clone( |ψ⟩ ) → |ψ⟩ |ψ⟩ )
7
8      # G3: Causality is respected
9      [] (forall measurement . timestamp(cause) < timestamp(
10          effect))
11
12     # G4: Measurement irreversibility
13     [] (collapsed(q) → not quantum(q))
14 }

```

Listing 21: Quantum Safety Invariants

11.2 Liveness Properties

```

1  PROPERTY quantum_liveness {
2      # L1: Every quantum state eventually decoheres
3      <> (forall q: QINT . coherence(q) < PLANCK_THRESHOLD)
4
5      # L2: Measurements eventually complete
6      (measure_initiated(q) → <> measure_complete(q))
7
8      # L3: Resources are eventually reclaimed
9      []<> (allocated_qubits = deallocated_qubits)
10
11     # L4: Entanglements eventually resolve
12     <> (forall entangled_pair . separated OR measured)
13 }

```

Listing 22: Quantum Liveness Guarantees

12 Performance Specifications

Performance Requirements

- **Quantum Coherence Time:** $\geq 1000\tau_{\text{planck}}$
- **State Preparation:** < 10 ns per qubit
- **Measurement Latency:** < 100 ns
- **Context Switch Overhead:** $< 1\mu\text{s}$
- **Memory Efficiency:** $O(n)$ for n qubits
- **Entanglement Depth:** Maximum 6 levels
- **Error Rate:** $< 10^{-9}$ per operation

13 Firmware Integration Guidelines

13.1 Git-RAF Integration Points

```

1  @firmware_integration {
2      git_raf_hooks: {
3          pre_commit: validate_quantum_governance()
4          post_merge: verify_collapse_consistency()
5          pre_push: audit_quantum_operations()
6      }
7
8      versioning: {
9          quantum_state_snapshot: on_commit
10         collapse_history: immutable_log

```



```

11         entanglement_graph: version_tracked
12     }
13
14     deployment: {
15         stage: [0..N+1]
16         firmware_target: quantum_coprocessor
17         validation_level: AEGIS_COMPLIANT
18     }
19 }

```

Listing 23: Git-RAF Firmware Hooks

14 Conclusion and Future Extensions

Stage-N of the RiftLang Protocol Stack provides a robust foundation for hybrid quantum-classical computation within the AEGIS governance framework. The specification enables:

- Seamless quantum-classical transitions
- Governance-compliant state management
- Planck-scale temporal constraints
- Integration with existing AEGIS phases
- Extensibility for future quantum algorithms

Future stages (N+1 and beyond) will extend this framework to support:

- Distributed quantum computation
- Fault-tolerant quantum error correction
- Advanced entanglement protocols
- Quantum machine learning integration

A Glossary of Terms

QINT Quantum Integer - A quantum register holding superposition states

Planck Time $\tau_{planck} = 5.39 \times 10^{-44}$ seconds

Coherence Quantum state phase relationship preservation

Collapse Quantum to classical state transition

AEGIS Automated Enterprise Governance Intelligence System

B References

1. AEGIS Project Technical Specification v2.0
2. RiftLang Compiler Documentation
3. Quantum Computing Governance Framework
4. Git-RAF Firmware Integration Manual
5. OBINexus Toolchain Architecture Guide